```
class Graph:
    def __init__(self):
        self.names = [
            "Computer",
            "AIML",
            "IT",
            "Mechanical",
            "Civil",
            "Electrical"
        ]

        self.graph = [
            [0, 10, 20, 0, 0, 0],
            [10, 0, 15, 30, 0, 0],
            [20, 15, 0, 0, 25, 0],
            [0, 30, 0, 0, 18, 20],
            [0, 0, 25, 18, 0, 12],
            [0, 0, 0, 20, 12, 0]
        ]

    def display(self):
        print("\nAdjacency Matrix:")
        for row in self.graph:
            print(row)

    def kruskal(self):
        edges = []

        for i in range(len(self.graph)):
            for j in range(i + 1, len(self.graph)):
                if self.graph[i][j] != 0:
                    edges.append((self.graph[i][j], i, j))

        edges.sort()

        parent = list(range(len(self.names)))

        def find(i):
            while parent[i] != i:
                i = parent[i]
            return i

        def union(i, j):
            parent[find(i)] = find(j)
```

```
    print("\nKruskal's MST:")
    total = 0
    count = 0

    for weight, u, v in edges:
        if find(u) != find(v):
            union(u, v)
            print(self.names[u], "--", weight, "m --", self.names[v])
            total += weight
            count += 1

            if count == len(self.names) - 1:
                break

    print("Total Distance =", total, "m")

def prim(self):
    n = len(self.names)
    selected = [False] * n
    selected[0] = True

    print("\nPrim's MST:")
    total = 0

    for _ in range(n - 1):
        minimum = float('inf')
        x = y = -1

        for i in range(n):
            if selected[i]:
                for j in range(n):
                    if not selected[j] and self.graph[i][j] != 0:
                        if self.graph[i][j] < minimum:
                            minimum = self.graph[i][j]
                            x = i
                            y = j

        selected[y] = True
        print(self.names[x], "--", minimum, "m --", self.names[y])
        total += minimum

    print("Total Distance =", total, "m")
```

```
g = Graph()

g.display()
g.kruskal()
g.prim()
```